

APPENDIX C - SDD



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING
UTM Johor Bahru

SECJ 3032: Software Engineering PSM1

Semester 01, 2024/2025

Software Design Descriptions (SDD)

Al-Muneer Online Portal

Version 1.2

Prepared by: Ahmed Ghaleb

Revision Page

a. Overview

This document is Version 1.2 of the Software Design Description (SDD) for the Al-Muneer Online Portal. It details the high-level and detailed design of the system, building upon the requirements defined in the Software Requirements Specification (SRS) Version 1.2. This SDD describes the system architecture, data design, interface design, component-level design, and AI integration patterns necessary to guide the implementation of the portal. This document is prepared as part of the SECJ 3032: Software Engineering FYP1 course requirements.

b. Issuing Organisation / Team

- Organisation: Faculty of Computing, Universiti Teknologi Malaysia (UTM)
- Team Name / Developer: Ahmed Hani Ahmed Ghaleb (Individual FYP1 Project)

c. Authorship / Project Team Members

Role	Team Member	Status of Assigned Task (for SDD V1.0)
Lead Analyst / Designer	Ahmed Hani Ahmed Ghaleb	Complete
Document Author	Ahmed Hani Ahmed Ghaleb	Complete
Technical Architect	Ahmed Hani Ahmed Ghaleb	Complete

d. **Revision History**

REVISION NO.	ISSUE DATE	DETAILS OF REVISION
00 (Draft)	15/05/2025	Initial internal draft based on SRS.
1.0	30/05/2025	First complete version of SDD for FYP1 Progress 2.
1.1	8/08/2025	Use case diagram updated: Deleted UC006.
1.2	18/06/2026	Synced with implementation: added Gemini AI advisor subsystem, asynchronous AI panel design, PageVisit tracking, reference-code booking flow, WhatsApp notification templates, and refined data model (GalleryLabel, maps URLs, corrected relationships).

Note:

This Software Design Descriptions (SDD template is adapted from IEEE Recommended Practice based on Software Design Descriptions (SDD) (IEEE Std. 10161998 1), that are simplified and customized to meet the need of SECJ3203 FYP1 SE course at Faculty of Computing, UTM. Examples of models are from Arlow and Neustadt (2002) and other sources stated accordingly.

Table of Contents

SECTION	TITLE	PAGE
<u>1. INTRODUCTION</u>		6
	<u>Purpose</u>	6
	<u>Scope</u>	7
	<u>Context</u>	8
	<u>Summary</u>	9
<u>2. REFERENCES</u>		10
<u>3. GLOSSARY</u>		11
<u>4. DESIGN BODY</u>		12
	<u>Design Stakeholders and Their Concerns</u>	13
	<u>Context Viewpoint</u>	15
	<u>Design Concerns</u>	16
	<u>Design View</u>	16
	<u>Composition Viewpoint</u>	17
	<u>Design Concerns</u>	18
	<u>Design View</u>	19
	<u>Logical Viewpoint</u>	20
	<u>Design Concerns</u>	20
	<u>Design View</u>	21
	<u>Information Viewpoint</u>	24
	<u>Design Concerns</u>	24
	<u>Design View</u>	25
	<u>Interface Viewpoint</u>	31
	<u>Design Concerns</u>	32
	<u>Structure Viewpoint</u>	37
	<u>Design Concerns</u>	37
	<u>Design View</u>	37
	<u>Interaction Viewpoint</u>	41
	<u>Design Concerns</u>	41
	<u>Design View</u>	42

SECTION	TITLE	PAGE
	State Dynamic Viewpoint	45
	Design Concerns	45
	Design View	46
	Algorithm Viewpoint	48
	Design Concerns	48
	Design View	49
<u>5. USER INTERFACE DESIGN</u>		52
	Overview of User Interface	52
	Screen Images (English Version)	54
<u>6. APPENDICES</u>		57

1. Introduction

The following sections of the Software Design Description (SDD) should provide the details of the entire document.

1.1 Purpose

This Software Design Description (SDD) document delineates the design for the Al-Muneer Online Portal. Its purpose is to translate the functional and non-functional requirements specified in the Software Requirements Specification (SRS) document (Version 1.2) into a detailed blueprint for implementation. This SDD describes the system architecture, data design, component design, interface design, AI integration design, and other design decisions that will guide the development of the portal.

This SDD describes how the Al-Muneer Online Portal will be structured and built. It is intended for use by the project developer (Ahmed Hani Ahmed Ghaleb) as the primary

technical guide for system implementation during PSM2, by the project supervisor for reviewing the technical design and ensuring it meets the requirements, and by examiners for assessing the design quality and completeness.

1.2 Scope

This SDD document covers the design of the "Al-Muneer Online Portal," a web-based application for Al-Muneer Hall for Weddings and Events. The scope of the design encompasses:

- **System Architecture:** The overall structure of the system, including its layers, components, AI service integration, and their interactions.
- **Database Design:** The logical and physical design of the relational database (PostgreSQL) that will store the portal's data, including entity definitions and relationships.
- **Component Design:** The design of major software components within the backend (Spring Boot) and frontend, detailing their responsibilities and interfaces, including the Gemini AI advisor service.
- **Interface Design:**
 - **User Interfaces (UI):** Conceptual design and layout for key user-facing screens for both Visitors and the Administrator, including asynchronous AI advisor panels in the admin dashboard.
 - **Internal Interfaces:** APIs between the frontend and backend components, including REST endpoints for AI insights.
- **Security Design Considerations:** How security requirements (authentication, authorisation, data protection, JWT cookie-based session management) will be incorporated into the design.
- **AI Integration Design:** How optional, non-blocking AI advisors (business, feedback, traffic funnel) are loaded asynchronously and degrade gracefully on API failure.

The software product will, as defined in the SRS:

- Allow visitors to view venue information, media, availability, pricing, submit booking inquiries with a 9-digit reference code, upload payment proofs, and provide feedback.

- Enable administrators to manage all content, inquiries, payment statuses, feedback, view traffic analytics and operational reports augmented by AI advisors, and configure WhatsApp notification templates.

The design will adhere to the constraints identified in the SRS, including the use of the specified technology stack (Spring Boot 3.4.4, Java 21, PostgreSQL, HTML/CSS/JS, Thymeleaf, Chart.js, Google GenAI Java SDK), deployment on a Cloud VPS, and considerations for the solo developer context and project timeline. This SDD aims to provide sufficient detail to implement a functional, reliable, and maintainable system as per the project's objectives.

The software product is a custom web application designed specifically for Al-Muneer Hall.

1.3 Context

This SDD document is intended to be used by:

- a) The **Project Developer (Ahmed Hani Ahmed Ghaleb)**, who will use this design as the primary guide for the implementation, coding, and unit testing of the Al-Muneer Online Portal during PSM2.
- b) The **Project Supervisor (Dr. Muhammad Luqman bin Mohd Shafie)**, who will use this document to review the technical soundness of the design, ensure its alignment with the specified requirements, and assess the progress and quality of the design work.
- c) **Academic Examiners**, who will use this document to evaluate the design aspects of the Final Year Project.

This standard (adapted IEEE Std. 1016-1998) is intended for technical and managerial stakeholders who prepare and use SDD. It guides the designer in the selection, organization, and presentation of design information. For an organization (in this case, the academic framework of an FYP) developing its own SDD practices, the use of this standard can help to ensure that design descriptions are complete, concise, consistent, interchangeable, appropriate

for recording design experiences and lessons learned, well organized, and easy to communicate.

1.4 Summary

This SDD document provides a comprehensive overview of the design for the Al-Muneer Online Portal.

- **Section 1-4:** Cover introductory material, references, and glossary.
- **Section 5 (Design Body):** This is the core section, detailing the design from multiple viewpoints as per the IEEE standard structure. It includes:
 - Design Stakeholders and Their Concerns (5.1)
 - Context Viewpoint (5.2): System boundary and external interactions (Use Case Diagram).
 - Composition Viewpoint (5.3): High-level system decomposition into major subsystems/modules (Architecture/Component Diagram).
 - Logical Viewpoint (5.4): Detailed class structure within subsystems (Class Diagrams).
 - Information Viewpoint (5.5): Persistent data design (ERD, Data Dictionary).
 - Interface Viewpoint (5.6): User, hardware, and software interface specifications.
 - Structure Viewpoint (5.7): Internal organization of components (Package Diagram).
 - Interaction Viewpoint (5.8): Dynamic behavior between components (Sequence Diagrams for key interactions).
 - State Dynamic Viewpoint (5.9): State changes of key entities (State Machine Diagrams, if applicable at a detailed level).
 - Algorithm Viewpoint (5.10): Logic for complex operations or algorithms, including AI prompt construction, reference-code generation, and status cascade rules.
- **Section 6 (User Interface Design):** Provides conceptual screen layouts and descriptions for key user interfaces

- **Section 7 (Appendices):** Includes any supporting design information.

This organization aims to present the design in a structured and understandable manner, covering static structures, dynamic behavior, data design, and interface specifications.

2. References

This section provides a complete list of all documents referenced elsewhere in this SDD.

1. Ahmed Hani Ahmed Ghaleb. (2026). *Software Requirements Specification (SRS) - Al-Muneer Online Portal, Version 1.2*. Universiti Teknologi Malaysia. (Internal Project Document)
2. Ahmed Hani Ahmed Ghaleb. (2026). *FYP1 Progress 2 Report - Al-Muneer Online Portal - Core Specifications & Implementation Changelog (README.md)*. Universiti Teknologi Malaysia. (Internal Project Document)
3. IEEE. (1998). *IEEE Recommended Practice for Software Design Descriptions* (IEEE Std 1016-1998). Institute of Electrical and Electronics Engineers.
4. Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional.
5. Google. (2026). Google GenAI Java SDK (google-genai 1.56.0). <https://github.com/googleapis/java-genai>

Sources for these documents include internal project documentation created as part of the FYP1 process, IEEE standards, established software engineering literature, and official documentation for the chosen technologies.

3. Glossary

This section provides the definitions of all key terms, acronyms, and abbreviations used in this SDD. Many of these are consistent with those defined in the SRS document.

- **Admin:** Administrator of the AI-Muneer Online Portal.
- **AI Advisor:** An optional, asynchronous insight panel powered by the Google GenAI Java SDK (Gemini) that analyses portal data and presents concise, actionable recommendations to the Administrator.
- **API:** Application Programming Interface.
- **Component Diagram:** A diagram that depicts how components are wired together to form larger components or software systems.
- **CRUD:** Create, Read, Update, Delete.
- **Deep Link:** A `wa.me` URL generated client-side from a WhatsApp template and a normalised phone number, allowing the admin to open a pre-filled WhatsApp chat.
- **ERD:** Entity-Relationship Diagram.
- **FAQ:** Frequently Asked Questions.
- **FYP:** Final Year Project. (Also **PSM**)
- **GenAI / Gemini:** Google's generative artificial intelligence service accessed through the official Google GenAI Java SDK; used by the system for optional business, feedback, and traffic insights.
- **JWT:** JSON Web Token — a compact, URL-safe token used for stateless admin authentication.
- **BCrypt:** A password-hashing function used to store administrator credentials securely.
- **HTML:** HyperText Markup Language.
- **IDE:** Integrated Development Environment.
- **IEEE:** Institute of Electrical and Electronics Engineers.
- **JS:** JavaScript.
- **MVC:** Model-View-Controller architectural pattern.
- **PageVisit:** A daily counter entity that records how many times a public visitor page was accessed.
- **PaaS:** Platform as a Service.

- **PSM:** Projek Sarjana Muda (Bachelor's Degree Project).
- **Reference Code:** A random 9-digit number assigned to each `BookingInquiry` and stored in a 150-day HTTP-only cookie so visitors can retrieve their inquiry.
- **REST:** Representational State Transfer – an architectural style for distributed hypermedia systems.
- **SDD:** System Design Document
- **SRS:** Software Requirements Specification.
- **SQL:** Structured Query Language.
- **SSL/TLS:** Secure Sockets Layer/Transport Layer Security.
- **UC:** Use Case.
- **UI:** User Interface.
- **UML:** Unified Modeling Language.
- **UX:** User Experience.
- **VPS:** Virtual Private Server.

4. Design Body

This section of the SDD details the design of the Al-Muneer Online Portal, addressing identified design stakeholders, their concerns, and presenting the design through various viewpoints. The selected design viewpoints are chosen to provide a comprehensive understanding of the system's structure, behavior, and data management, based on the rationale of clarity and completeness for the implementation phase.

4.1 Design Stakeholders and Their Concerns

This section specifies the design stakeholders for the Al-Muneer Online Portal and the primary design concerns of each. An SDD shall address each identified design concern.

a) **The design stakeholders for the design subject (Al-Muneer Online Portal) are:**

Mr. Ahmed Almunajid (Venue Owner/Primary User of Admin Panel): Concerned with the ease of use of the admin panel, the accuracy of information displayed, the efficiency of managing inquiries and bookings, the reliability of the system, and how well it meets his business needs to reduce manual workload and improve customer interaction.

Ahmed Hani Ahmed Ghaleb (Student Developer): Concerned with the clarity of the design for implementation, the feasibility of building the system with the chosen technologies within the given timeframe, the maintainability of the code, adherence to academic requirements, and the overall technical quality of the solution.

Prospective Clients/Visitors: Concerned with the ease of finding information, the clarity and accuracy of content (availability, pricing, venue details), the simplicity of submitting inquiries and payment proofs, and a positive, trustworthy user experience.

*** Project Supervisor & Academic Examiners:** Concerned with the technical soundness of the design, its completeness in addressing the requirements from the SRS, adherence to good software engineering principles, proper documentation, and the overall quality of the project as an academic endeavor.

b) **The design concerns of each identified design stakeholder include: Mr. Ahmed Almunajid:**

*** Usability:** How easy will it be for him to update content, manage bookings, and use the admin features daily?

*** Functionality:** Will the system reliably perform all the required tasks (calendar updates,

inquiry management, payment proof handling, feedback viewing, report generation, notifications)?

* **Data Integrity & Accuracy:** Will the information shown to visitors be accurate and up-to-date? Will booking and payment information be stored reliably?

* **Efficiency:** Will the portal genuinely save time compared to current manual methods?

* **Security (Admin Access):** Is his access to the admin panel secure?

* **Ahmed Hani Ahmed Ghaleb (Developer): Clarity & Completeness:** Is the design detailed enough to guide implementation without ambiguity?

* **Modularity & Maintainability:** Is the system designed in a way that is easy to code, test, and potentially modify or extend later?

* **Feasibility:** Can all designed aspects be realistically implemented with the chosen technology stack (Spring Boot 3.4.4, Java 21, PostgreSQL 16, HTML/CSS/JS, Thymeleaf, Chart.js, Google GenAI Java SDK) and within the project timeline?

Performance: Will the design support the required response times and handle the anticipated load?

* **Security:** Are appropriate security measures designed into the system (input validation, secure data handling, authentication)?

Prospective Clients/Visitors:

Ease of Use: Can they find what they need quickly and intuitively?

Information Quality: Is the information accurate, comprehensive, and clearly presented?

Responsiveness: Does the site load quickly and respond well on different devices?

Trust & Reliability: Does the site appear professional and trustworthy, especially when submitting personal information for inquiries or payment proofs?

Accessibility: Is the site usable across different browsers and devices?

Project Supervisor & Academic Examiners:

Requirement Coverage: Does the design adequately address all functional and non-functional requirements specified in the SRS?

Design Soundness: Are appropriate architectural patterns, data models, and design principles applied?

Documentation Quality: Is the design well-documented, clear, and consistent?

Technical Depth: Does the design demonstrate a sufficient understanding of software engineering concepts?

Adherence to Constraints: Does the design respect the defined project constraints (technology, timeline, etc.)?

This SDD aims to address these concerns by providing a clear, detailed, and justified design for the Al-Muneer Online Portal.

4.2 Context Viewpoint

The Context viewpoint depicts services provided by a design subject (the Al-Muneer Online Portal) with reference to an explicit context. That context is defined by reference to actors that include users and other stakeholders, which interact with the design subject in its environment. The Context viewpoint provides a “black box” perspective on the design subject.

4.2.1 Design Concerns

This section should specify the following:

- a) The purpose of the Context viewpoint is to identify:
 - The system (Al-Muneer Online Portal) as a whole.
 - The actors that interact with the system (Visitors, Administrator).
 - The interactions (services offered/used) between the actors and the system.
 - The boundary of the system, distinguishing what is internal and external.
- b) The system features from an external "black box" perspective.

The purpose of the Context viewpoint is to identify a design subject's offered services, its actors (users and other interacting stakeholders), to establish the system boundary and to effectively delineate the design subject's scope of use and operation. This is primarily achieved through a Use Case Diagram which illustrates the services (use cases) the system provides to its different actors.

4.2.2 Design View

The system features, from a context viewpoint, are represented by the services it offers to its external actors. These are best illustrated by the Use Case Diagram, which outlines the scope of operations for the Al-Muneer Online Portal.

The system features include:

- For **Visitors**: Viewing venue details, browsing the media gallery, checking availability via an interactive calendar, viewing pricing information, submitting booking inquiries with a 9-digit reference code, optionally submitting proof of payment, and providing feedback.
- For the **Administrator**: Securely logging in to manage all website content (hall information, media, availability calendar, pricing, FAQs), managing booking

inquiries, managing submitted payment proofs and their statuses, reviewing user feedback, viewing traffic analytics and operational reports augmented by AI advisors, and configuring WhatsApp notification templates.

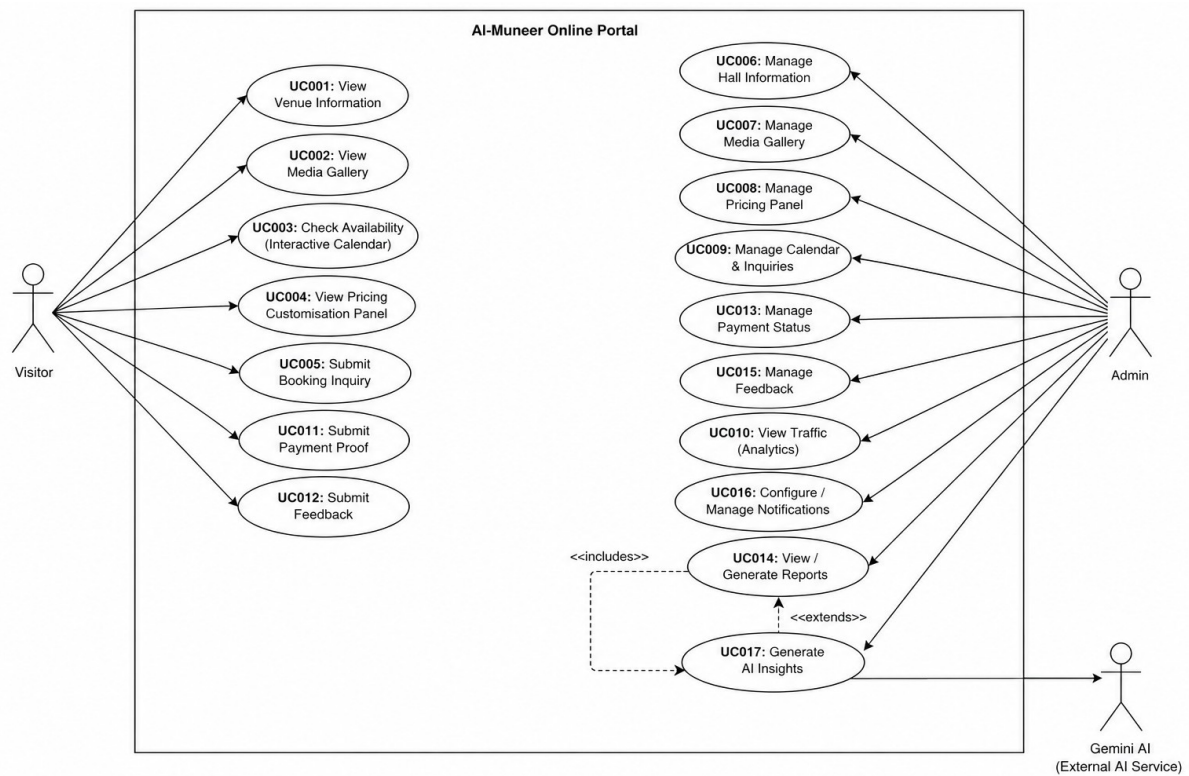


Figure 5.1: Use Case Diagram for AI-Muneer Online Portal

4.3 Composition Viewpoint

The Composition viewpoint describes the way the design subject (AI-Muneer Online Portal) is (recursively) structured into constituent parts and establishes the roles of those parts. It provides a high-level overview of the system's internal organization.

4.3.1 Design Concerns

This should specify the following:

- a) **The major design constituents (subsystems/modules):** Identifying the primary high-level blocks that make up the system.
- b) **The subsystem responsibilities:** Defining the main role or purpose of each identified subsystem/module.

The purpose of the composition viewpoint is to define the subsystems that compose the system. The AI-Muneer Online Portal is formed by several interacting modules that logically group related functionalities. This decomposition aids in understanding the system's structure, assigning development responsibilities (though a solo project, it helps in organizing work), and managing complexity.

The system formed by these modules aims to deliver the functionalities outlined in the SRS. For the AI-Muneer Online Portal, we can identify the following major logical subsystems:

1. **Presentation Subsystem (Frontend):** Responsible for all user interaction, rendering information, and capturing user input. This subsystem itself can be seen as having two main parts: the Public-Facing Interface and the Administrator Panel Interface. It loads AI advisor panels asynchronously after the initial page render.
2. **Application Logic Subsystem (Backend Core):** Responsible for implementing the core business logic, processing requests from the frontend, interacting with the data layer, and managing application state.
3. **Data Management Subsystem (Backend Persistence):** Responsible for all aspects of data storage and retrieval, ensuring data integrity and consistency.
4. **Notification Subsystem (Backend Service):** Responsible for managing WhatsApp message templates and generating client-side deep links for visitor communication.
5. **Reporting Subsystem (Backend Service):** Responsible for generating operational reports and aggregating data for visual charts and AI advisors.
6. **Analytics Subsystem (Backend Service):** Responsible for tracking public page visits and serving traffic analytics data to the admin dashboard and AI funnel advisor.

7. **AI/GenAI Subsystem (Backend Service):** Responsible for constructing prompts from portal data and calling the Google GenAI Java SDK (Gemini) to generate optional business, feedback, and traffic insights. This subsystem is designed to be non-blocking and to return graceful fallbacks on failure.

4.3.2 Design View

This section develops a component model and explains the relationships between the components to achieve the complete functionality of the system. This is a high-level overview of how responsibilities of the system were partitioned and then assigned to subsystems.

The Al-Muneer Online Portal is decomposed into the following major subsystems/components, which collaborate to deliver the required functionalities. Figure 5.2 provides a high-level component diagram illustrating these major subsystems and their primary interactions.

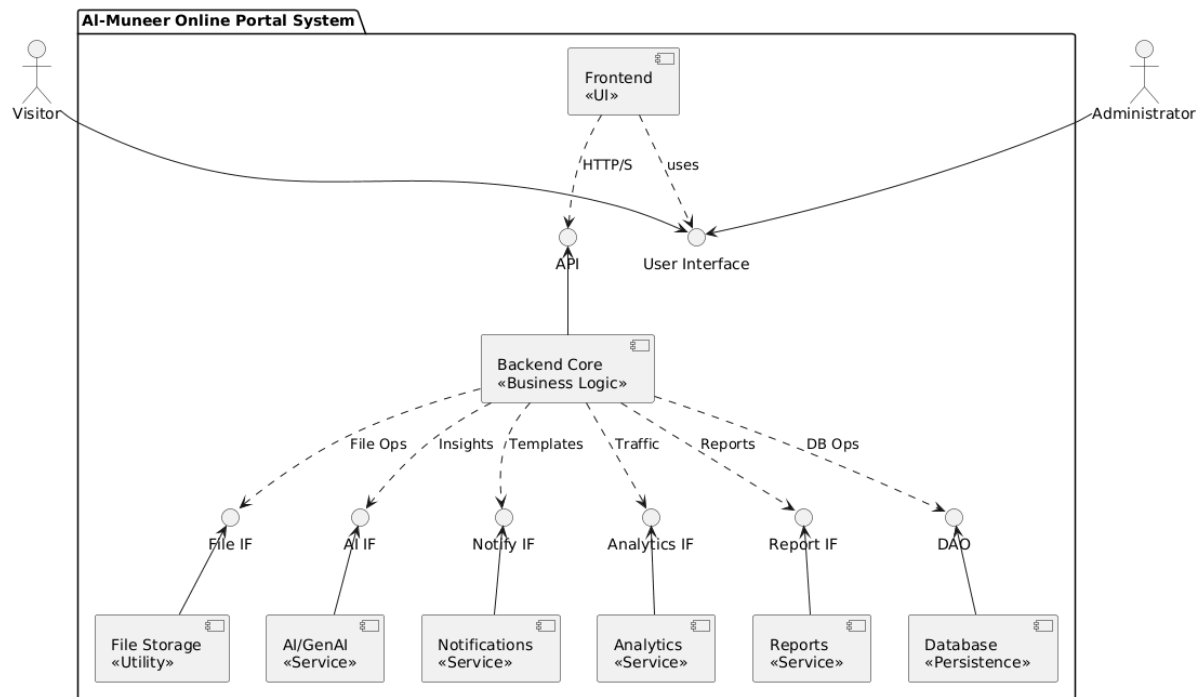


Figure 5.2: Component Diagram of AI-Muneer Online Portal Subsystems

4.4 Logical Viewpoint

The Logical Viewpoint elaborates on existing and designed types and their implementations as classes and interfaces, along with their structural static relationships. This viewpoint provides a detailed look at the internal structure of the software components identified in the Composition Viewpoint, particularly focusing on the organization of classes within the Application Logic Subsystem (Backend Core).

4.4.1 Design Concerns

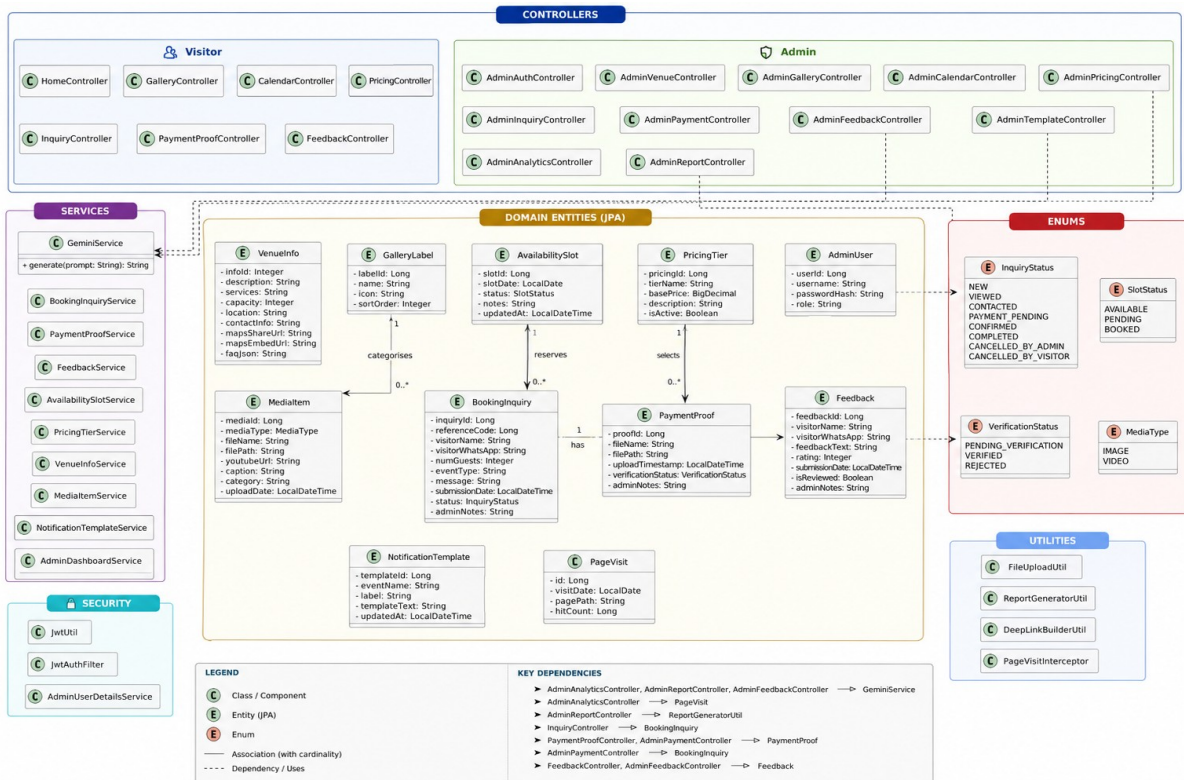
This should specify the following:

- a) **The classes and interfaces:** Identifying key classes and interfaces that realize the system's functionality.
- b) **The relationships:** Describing the static relationships between these classes and interfaces (e.g., association, aggregation, inheritance).

The purpose of the logical viewpoint is to elaborate the implementation of classes with their relationships. It focuses on the functional requirements of the system by describing the main classes and their responsibilities within the domain. For the Al-Muneer Online Portal, this involves detailing the key entities, controllers, services, and repositories that constitute the backend system.

4.4.2 Design View

This design view includes a class diagram representing key classes within the respective subsystems/packages, particularly focusing on the controller and domain entity classes for the Al-Muneer Online Portal's backend. The Spring Boot framework encourages a layered architecture often comprising Controller, Service, and Repository layers for each primary domain entity.



Class Diagram for Key Backend Entities, Controllers, and AI Service (Conceptual)

The backend design primarily follows the Model-View-Controller (MVC) layered pattern, where Spring Boot facilitates this structure.

- **Controller Classes:** Stereotyped as Spring `@Controller` or `@RestController`, these handle incoming HTTP requests from the Presentation Subsystem (Frontend), parse request data, delegate business logic processing to Service classes, and return HTTP responses or Thymeleaf view names. Visitor-facing controllers (e.g., `InquiryController`, `FeedbackController`) serve HTML pages, while admin controllers (e.g., `AdminReportController`, `AdminAnalyticsController`) manage the protected admin panel and expose asynchronous AI insight endpoints.
- **AI Service ('GeminiService'):** A dedicated Spring `@Service` that wraps the Google GenAI Java SDK. It accepts a plain-text prompt, calls the Gemini model, and returns generated text. The service is designed to be resilient: any SDK or network failure is caught and a human-readable fallback message is returned, ensuring that AI panels never block page rendering.

- **Domain Entity Classes** (e.g., `VenueInfo`, `BookingInquiry`, `PaymentProof`, `PageVisit`): These are Plain Old Java Objects (POJOs) annotated as JPA `@Entity`'s. They represent the persistent data structures of the application and are mapped to database tables. Relationships between entities (e.g., a `BookingInquiry` references one `AvailabilitySlot` and optionally one `PricingTier`; a `BookingInquiry` may have many `PaymentProof`'s) are defined here.
- **Data Transfer Objects (DTOs)**: Simple objects used to transfer data between layers, particularly between the Controller layer and the client (Frontend), and sometimes between Service and Controller. They help decouple the service layer from the presentation layer.
- **Service Classes**: Stereotyped as Spring `@Service`'s (with interfaces and `impl` implementations), these contain the core business logic. Controllers delegate calls to them. For instance, `BookingInquiryService` handles saving a new inquiry, generating a random `referenceCode`, and freeing a slot on visitor cancellation; `PaymentProofService` handles the status cascade from proof verification to inquiry and slot.
- **Repository Classes (Implied, not shown in diagram for brevity)**: Interfaces extending Spring Data JPA (`JpaRepository`) stereotyped as Spring `@Repository`'s. They provide CRUD operations and custom query methods for the domain entities.
- **Security Classes** (`JwtUtil`, `JwtAuthFilter`, `AdminUserDetailsService`): Implement stateless JWT cookie-based authentication for the admin panel, using BCrypt for password hashing.

The relationships shown in the diagram are primarily conceptual associations indicating which controllers manage which entities (via the service and repository layers). The ERD in the Information Viewpoint (Section 5.5) provides a database-centric view of entity relationships. This logical view ensures separation of concerns, loose coupling, and maintainability.

4.5 Information Viewpoint

This section explains how the information domain of the Al-Muneer Online Portal is transformed into data structures. It describes how the major data or system entities are stored, processed, and organized. The primary data storage will be a relational database (PostgreSQL). This database will consist of several tables, where each table represents an entity and stores its attributes.

The major data or system entities are stored in a relational database named `almuneer\_portal\_db`. This database comprises the following key entities, which are detailed further in the Data Dictionary and ERD.

4.5.1 Design Concerns

This section explains how the information domain of the Al-Muneer Online Portal is transformed into data structures. It describes how the major data or system entities are stored, processed, and organized. The primary data storage will be a relational database (MySQL/PostgreSQL). This database will consist of several tables, where each table represents an entity and stores its attributes.

The major data or system entities are stored in a relational database named `almuneer_portal_db` (example name). This database comprises the following key entities, which are detailed further in the Data Dictionary and ERD.

Entity Name List (Conceptual):

1. AdminUser
2. VenueInfo
3. MediaItem
4. GalleryLabel
5. AvailabilitySlot
6. PricingTier
7. BookingInquiry
8. PaymentProof
9. Feedback
10. NotificationTemplate
11. PageVisit

4.5.2 Design View

Stores the single record of hall description, services, capacity, contact details, Google Maps URLs, and FAQ JSON.

Data Dictionary

Entity: AdminUser

Manages administrator login credentials and roles.

Attribute Name	Type	Description
userId	BIGINT	Unique identifier for the user
username	VARCHAR(50)	Login username
passwordHash	VARCHAR(255)	Hashed password
role	VARCHAR(20)	User role (e.g., "ADMIN")

Entity: VenueInfo

Attribute Name	Type	Description
infolId	INT	Unique identifier (single-row configuration table)
description	TEXT	Detailed description of the hall
services	TEXT	List of services offered
capacity	INT	Maximum guest capacity
location	VARCHAR	Physical address label of the hall
contactInfo	VARCHAR	Contact details
mapsShareUrl	TEXT	Google Maps share/open link
mapsEmbedUrl	TEXT	Google Maps embed iframe URL
faqJson	JSON or TEXT	FAQs stored as JSON

Entity: MediaItem

A gallery image or video. Local files are served from `/uploads/gallery/\*\*`; videos may also be embedded from YouTube.

Attribute Name	Type	Description
mediaId	BIGINT	Unique identifier for the media item
mediaType	VARCHAR(12)	Type of media ("image", "video")
fileName	VARCHAR	Original name of the uploaded file
filePath	VARCHAR	Optional caption for the media
youtubeUrl	VARCHAR	URL for embedded YouTube videos
caption	VARCHAR	Optional caption for the media
category	VARCHAR(30)	Gallery filter label (foreign key concept to GalleryLabel)
uploadDate	TIMESTAMP	Date and time of upload

Entity: AvailabilitySlot

Visitor-facing category used by the gallery filter bar.

Attribute Name	Type	Description
labelId	BIGINT	Unique identifier
name	VARCHAR(60)	Display name of the label (unique)
icon	VARCHAR(8)	Optional emoji/icon prefix
sortOrder	INT	Display order in the filter bar

Entity: AvailabilitySlot

Represents a calendar date and its booking status. Future dates default to `AVAILABLE`; past dates are dimmed in the UI.

Attribute Name	Type	Description
slotId	BIGINT	Unique identifier for the slot
slotDate	DATE	The specific date (unique)
status	VARCHAR	Booking status (`AVAILABLE`, `PENDING`, `BOOKED`)
notes	TEXT	Internal notes for the admin regarding this slot
updatedAt	TIMESTAMP	Last update timestamp

Entity: PricingTier

A pricing package that visitors can view and pre-select when submitting an inquiry.

Attribute Name	Type	Description
pricingId	BIGINT	Unique identifier for the tier
tierName	VARCHAR	Name of the pricing tier/package
basePrice	DECIMAL	Base price for this tier

description	TEXT	Details of what the tier includes
isActive	BOOLEAN	Whether this tier is currently active

Entity: BookingInquiry

A visitor's booking request, linked to a calendar slot and optionally a pricing tier. Each inquiry receives a random 9-digit `referenceCode` used for visitor self-service lookup and cancellation.

Attribute Name	Type	Description
inquiryId	BIGINT	Unique identifier for the inquiry
referenceCode	BIGINT	Random 9-digit visitor-facing reference code (unique)
visitorName	VARCHAR	Name of the person making the inquiry
visitorWhatsApp	VARCHAR(50)	Normalised WhatsApp contact number of the visitor
numGuests	INT	Estimated number of guests
eventType	VARCHAR	Type of event (e.g., Wedding, Gathering)
pricing_id	BIGINT	Foreign key linking to the selected PricingTier (nullable)
slot_id	BIGINT	Foreign key linking to the desired AvailabilitySlot
message	TEXT	Specific questions or requirements
submissionDate	TIMESTAMP	Date and time inquiry was submitted
status	VARCHAR(50)	Status ('NEW', 'VIEWED', 'CONTACTED', 'PAYMENT_PENDING', 'CONFIRMED', 'COMPLETED', 'CANCELLED_BY_ADMIN', 'CANCELLED_BY_VISITOR')

adminNotes	TEXT	Internal notes by admin for this inquiry
------------	------	--

Entity: PaymentProof

An uploaded receipt/screenshot attached to a booking inquiry. Verifying a proof cascades status changes to the linked inquiry and slot.

Attribute Name	Type	Description
proofId	BIGINT	Unique identifier for the proof
inquiry_id	BIGINT	Foreign key linking to BookingInquiry
fileName	VARCHAR	Original name of the uploaded file
filePath	VARCHAR	Server path where the file is stored
uploadTimestamp	TIMESTAMP	Date and time of upload
verificationStatus	VARCHAR	Status (`PENDING_VERIFICATION`, `VERIFIED`, `REJECTED`)
adminNotes	TEXT	Internal notes by admin for this proof

Entity: Feedback

Visitor-submitted review with a mandatory rating and message. Name and WhatsApp are optional.

Attribute Name	Type	Description
feedbackId	BIGINT	Unique identifier for the feedback
visitorName	VARCHAR	Optional name of the person providing feedback
visitorWhatsApp	VARCHAR	Optional WhatsApp contact details
feedbackText	TEXT	The feedback message itself
rating	INT	Rating (1-5 stars)
submissionDate	TIMESTAMP	Date and time feedback was

		submitted
isReviewed	BOOLEAN	Whether admin has reviewed this feedback
adminNotes	TEXT	Internal notes by admin for this feedback

Entity: NotificationTemplate

Database-backed WhatsApp message templates. Placeholders (`{visitorName}`, `{inquiryId}`, `{eventDate}`, `{status}`) are resolved client-side before generating a `wa.me` deep link.

Attribute Name	Type	Description
templateId	BIGINT	Unique identifier for the template
visitorName	VARCHAR	Optional name of the person providing feedback
eventName	VARCHAR	Machine-readable name/trigger (e.g., <code>PAYMENT_VERIFIED</code>)
label	VARCHAR(255)	Human-readable label shown in the admin UI dropdown
templateText	TEXT	The actual message content containing placeholders
updatedAt	TIMESTAMP	Last update timestamp of the template

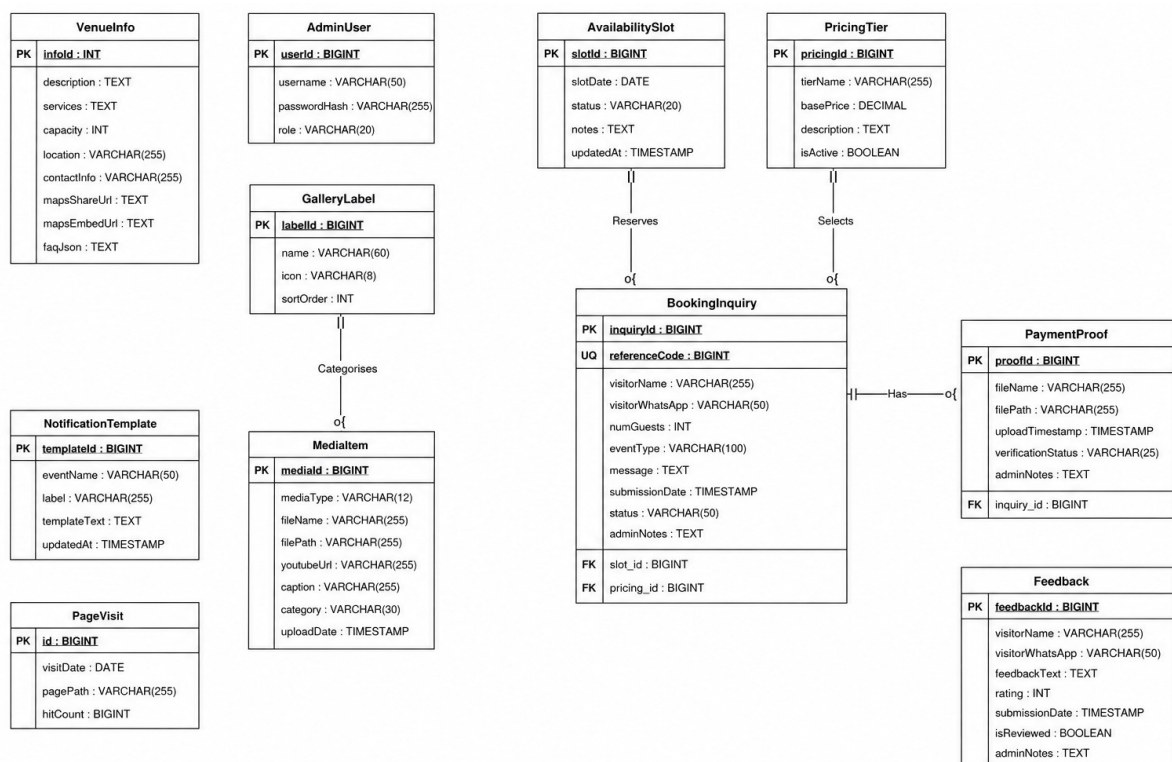
Entity: PageVisit

Daily hit counter for public visitor pages. Each row records one page on one date.

Attribute Name	Type	Description
id	BIGINT	Unique identifier

visitDate	DATE	Date of the visit
pagePath	VARCHAR(255)	Visited URL path (e.g., '/', '/gallery')
hitCount	BIGINT	Number of hits for that page on that date

Entity Relationship Diagram (ERD)



4.6 Interface Viewpoint

The Interface Viewpoint provides information for designers, programmers, and testers to correctly use the services provided by the Al-Muneer Online Portal and for the portal to interact with other entities. This description includes details of external and internal

interfaces. This viewpoint consists of a set of interface specifications for each entity that interacts with the system or for interfaces between major system components.

4.6.1 Design Concerns

This section describes the functionality of the system from the user's perspective and explains how users will interact with the system to complete expected features. It also covers interfaces with other hardware or software. For the Al-Muneer Online Portal, the interface viewpoint primarily covers User Interfaces (UI) for Visitors and the Administrator, and Software Interfaces (APIs) between the frontend and backend. Direct hardware interfaces are minimal for a web application beyond standard peripheral use.

4.6.2 Design View

User Interfaces

This specifies the logical characteristics of each interface between the software product and its users, including configuration characteristics necessary to accomplish the software requirements, and aspects of optimizing the interface.

- a) **Logical Characteristics:**
The Al-Muneer Online Portal will feature a web-based graphical user interface (GUI) accessible via standard web browsers.
- * **Visitor Interface:**
- * **Screen Layouts:** Will consist of clear, well-organized pages for Homepage, Venue Information, Media Gallery, Availability Calendar, Pricing, FAQ, Booking Inquiry, Payment Proof Submission, and Feedback. Navigation will be consistent via a main menu.
 - * **Content:** Will display textual information, images, videos, an interactive calendar, and forms for user input.
 - * **Interaction:** Primarily through mouse clicks, touch on touch-enabled devices, and keyboard input for forms.

* **Administrator Interface (Admin Panel):**

- * **Screen Layouts:** A secure, dashboard-style interface with sections for managing venue content, availability, pricing, inquiries, payment proofs, feedback, traffic analytics, reports, and WhatsApp notification templates. Will heavily utilize forms, tables for data display, action buttons, and asynchronous AI advisor panels.
- * **Content:** Will display system data in manageable formats, configuration options, Chart.js visualisations, AI-generated insight cards, and input fields for updates.
- * **Interaction:** Secure JWT cookie-based login required. Interaction via mouse clicks and keyboard input for data entry and management tasks. AI panels load asynchronously after the page renders and display a spinner until the insight is returned.

b) **Optimizing the Interface:**

- * **Simplicity:** Interfaces will be kept clean and uncluttered, avoiding unnecessary complexity, especially for the admin panel to ensure ease of use for Mr. Almunajid.
- * **Consistency:** Navigation, terminology, and layout will be consistent across the portal.
- * **Feedback:** The system will provide immediate visual feedback for user actions (e.g., successful form submission, error messages). Error messages will be informative.
- * **Efficiency:** Forms will be designed to minimize clicks and data entry effort.
- * **Language:** Primarily Arabic, with consideration for English in key navigational elements, aligning with user characteristics.

Hardware Interfaces

This specifies the logical characteristics of each interface between the software product and the hardware components of the system. For the Al-Muneer Online Portal, a web-based application, there are no direct, custom hardware interfaces designed into the software itself. The system will rely on standard user hardware:

- **Client-side:** User's computer (desktop, laptop), tablet, or smartphone with a web browser and internet connection. Standard input devices (keyboard, mouse, touchscreen) are assumed.
- **Server-side:** The Cloud VPS will have standard server hardware (CPU, RAM, storage, network interface) managed by the hosting provider. The application will

interact with this hardware through the operating system and the Java Virtual Machine.

Software Interfaces

This specifies the use of other required software products and interfaces with other application systems.

1. Operating Systems:

- **Client-Side:** The portal will be accessible from standard operating systems that support modern web browsers (e.g., Windows, macOS, Linux, Android, iOS).
- **Server-Side:** The Cloud VPS will run a common server operating system (e.g., a Linux distribution like Ubuntu).
- **Interface:** Standard interactions via web browser (client) and JVM/OS system calls (server).

2. Web Server:

- The Spring Boot application includes an embedded web server (e.g., Tomcat, Jetty, or Undertow by default).
- **Interface:** HTTP/HTTPS protocols for communication with client browsers.

3. Database Management System (DBMS):

- PostgreSQL 16.
- **Interface:** The Spring Boot application will interface with the DBMS using JDBC and Spring Data JPA for data persistence and retrieval.

4. Web Browsers (Client-Side):

- Google Chrome, Mozilla Firefox, Apple Safari, Microsoft Edge (latest stable versions).
- **Interface:** The system outputs HTML (rendered by Thymeleaf), CSS, and JavaScript. It receives user input via HTTP requests. Admin pages fetch AI insights asynchronously via `fetch()` to `/admin/reports/ai-insight``, `/admin/feedback/ai-insight``, and `/admin/analytics/ai-insight``.

5. AI/GenAI Service:

- Google GenAI Java SDK (`google-genai` 1.56.0) with model `gemini-3.1-flash-lite`.
- **Interface:** `GeminiService.generate(prompt)` sends a plain-text prompt to the Gemini API and returns generated text. The service is called from admin AI controllers and returns graceful fallback text on any failure.

6. WhatsApp Deep Links:

- Client-side JavaScript resolves `NotificationTemplate` placeholders (e.g., `{visitorName}`, `{inquiryId}`, `{eventDate}`, `{status}`) against the current record and opens a `wa.me` link with the composed message.
- **The underlying network protocol suite for all internet-based communication:** No server-side WhatsApp API integration; the browser constructs the deep link using the configured `app.country.code` and the normalised visitor phone number.

Communication Interfaces

This specifies the various interfaces to communications such as local network protocols, etc.

- **HTTP/HTTPS:** The primary communication protocol between the client web browser and the web server hosting the Al-Muneer Online Portal. HTTPS will be enforced for secure data transmission.
- **TCP/IP:** The underlying network protocol suite for all internet-based communication.
- **JDBC:** Used for communication between the Java application (Spring Boot backend) and the PostgreSQL database.
- **External HTTPS:** Used by `GeminiService` to call the Google GenAI REST API.

Key Internal API Patterns

Representative server-rendered and API routes that realise the design include:

- Visitor routes: `GET /`, `GET /gallery`, `GET /calendar`, `GET /pricing`, `GET /inquiry`, `POST /inquiry/submit`, `GET /inquiry/confirmation/{refCode}`, `POST /inquiry/cancel`, `POST /payment/upload`, `POST /feedback/submit`.
- Admin routes: `GET /admin/login`, `POST /admin/login`, `GET /admin/dashboard`, `GET /admin/inquiries`, `GET /admin/payments`, `POST

`/admin/payments/{id}/verify``, ``GET /admin/calendar``, ``GET /admin/reports``,
``GET /admin/analytics``, ``GET /admin/feedback``, ``GET /admin/templates``.

- Asynchronous AI insight endpoints: ``GET /admin/reports/ai-insight``, ``GET /admin/feedback/ai-insight``, ``GET /admin/analytics/ai-insight``.

All admin routes except ``/admin/login`` require a valid ``jwt`` HTTP-only cookie.

4.7 Structure Viewpoint

The Structure Viewpoint is used to document the internal constituents and organization of the design subject (Al-Muneer Online Portal) in terms of like elements (recursively). It informs the user and the developer about the interaction between the packages in the system and how the software is organized into modules or components at a finer granularity than the Composition Viewpoint.

4.7.1 Design Concerns

The primary design concerns for the Structure Viewpoint are:

- **Modularity:** How the system is broken down into manageable, cohesive, and loosely coupled packages or modules.
- **Dependencies:** The relationships and dependencies between these packages.
- **Organization:** The overall architectural organization of the code within the backend application.
- **Reusability:** Identifying components or packages that might be reusable.
- **Maintainability:** Ensuring the structure supports ease of understanding and modification.

Structure viewpoint informs user and the developer about the interaction between the packages in the system. For the Al-Muneer Online Portal, the backend developed using Spring Boot will be organized into several key packages, each with specific responsibilities, reflecting a layered architecture and separation of concerns.

4.7.2 Design View

This section includes the overall package diagram of the system, primarily focusing on the backend (Spring Boot application) structure. The navigation visibility is based on the dependency among classes in the design class diagram (Logical Viewpoint).

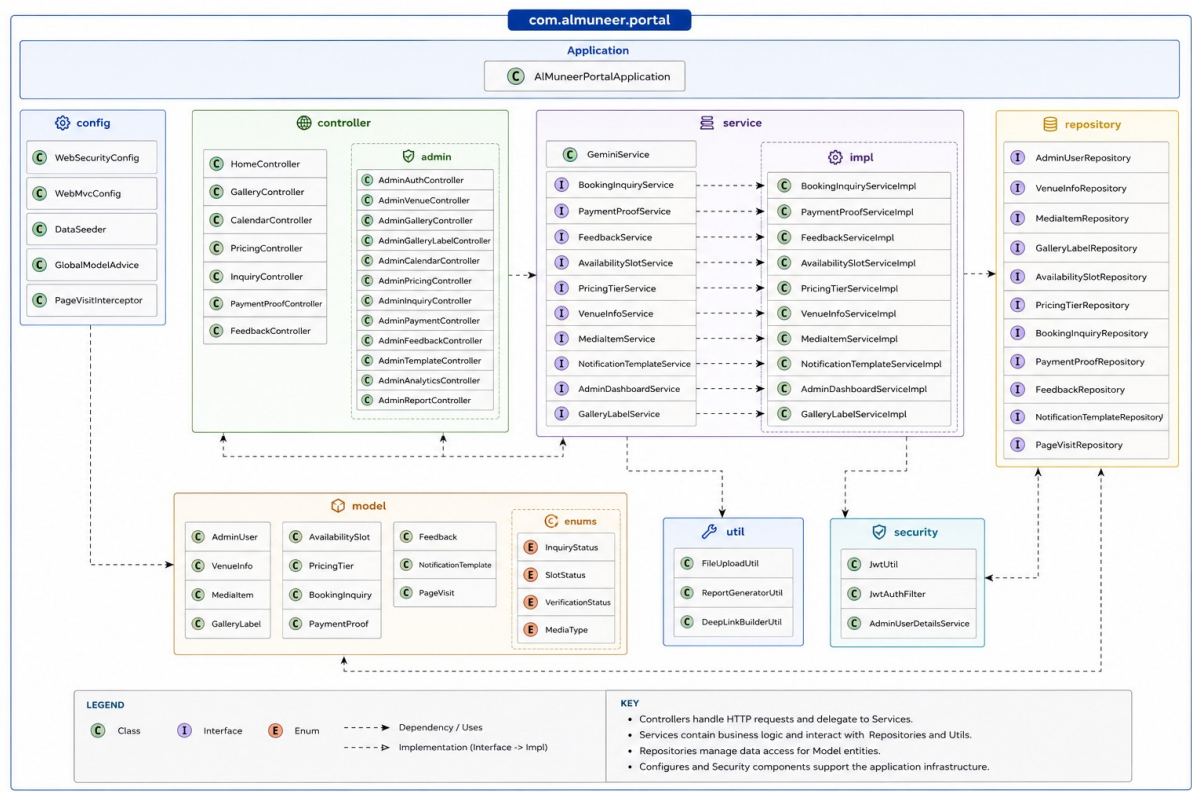


Figure 5.5: Package Diagram for AI-Muneer Online Portal Backend (click on it to expand)

Description of Backend Packages:

The backend of the AI-Muneer Online Portal is developed using Spring Boot and follows a layered architecture that promotes modularity, maintainability, and a clear separation of concerns. The application is organized into several packages, each serving a specific responsibility within the system.

At the root level, the `com.almuneer.portal` package contains the main application class, `AlMuneerPortalApplication`, which serves as the entry point of the system and is responsible for bootstrapping the Spring Boot application.

The `config` package contains configuration classes that define system-wide settings and behaviors. The `WebSecurityConfig` class configures Spring Security, including JWT cookie authentication, authorization rules, BCrypt password hashing, and protected route definitions. The `WebMvcConfig` class manages Spring MVC configurations, including static resource handlers for uploaded files and registration of the `PageVisitInterceptor`. The `GlobalModelAdvice` class provides global model attributes, such as the country code, that are automatically available to all Thymeleaf templates. The `DataSeeder` component initializes the system with a default administrator account and sample data during the first application run. Additionally, the `PageVisitInterceptor` tracks public visitor requests and updates the daily page visit statistics used for analytics purposes.

The `controller` package represents the presentation and request-handling layer of the application. It contains Spring MVC controller classes responsible for processing incoming HTTP requests and returning either Thymeleaf views or JSON responses. Public-facing controllers such as `HomeController`, `InquiryController`, and `FeedbackController` manage interactions with website visitors. The `controller.admin` sub-package contains administrator-specific controllers, including `AdminReportController`, `AdminAnalyticsController`, and `AdminFeedbackController`, which manage administrative functions and provide access to AI-generated business insights.

The `service` package implements the business logic layer of the system. It contains service interfaces that define business operations and an `impl` sub-package containing their concrete implementations. Service interfaces such as `BookingInquiryService` and `PaymentProofService` establish contracts for application functionality, while implementation classes such as `BookingInquiryServiceImpl` and `PaymentProofServiceImpl` contain the core business logic. These classes coordinate interactions between repositories, utility classes, and other services while enforcing business rules and managing transactions. The package also includes the `GeminiService`, a dedicated AI integration service that encapsulates communication with the Google GenAI Java SDK and provides AI-powered analytical capabilities within the administration module.

The **repository** package forms the data access layer of the application. It contains Spring Data JPA repository interfaces such as **InquiryRepository** and **AdminUserRepository**, which provide database access functionality. By extending Spring Data interfaces such as **JpaRepository**, these repositories automatically support standard CRUD operations while also allowing the implementation of custom query methods. This approach minimizes boilerplate code and simplifies interaction with persistent data stored in the system database.

The **model** package contains the domain entities that represent the application's persistent data structures. These Plain Old Java Objects (POJOs), including classes such as **BookingInquiry**, **AdminUser**, and **PageVisit**, are typically annotated with JPA annotations such as **@Entity** to establish mappings between Java objects and database tables. The package also contains a **model.enums** sub-package that stores enumerated types used throughout the system, including **InquiryStatus**, **SlotStatus**, **VerificationStatus**, and **MediaType**. These enumerations ensure consistent representation of predefined values within both the application logic and database.

Supporting functionality is grouped within the **util** package, which contains reusable utility classes shared across multiple system components. The **FileUploadUtil** class manages secure file uploads for gallery images and payment proof submissions. The **ReportGeneratorUtil** class aggregates inquiry, payment proof, and feedback data into structured report models that support both reporting features and AI-generated business recommendations. The **DeepLinkBuilderUtil** class assists in formatting WhatsApp phone numbers and message content for client-side deep-link generation.

Application security concerns are encapsulated within the **security** package. This package contains classes that support JWT-based authentication and authorization using Spring Security. The **JwtUtil** class is responsible for generating and validating JWT tokens using a configurable application secret. The **JwtAuthFilter** processes incoming requests by reading JWT tokens stored within HTTP-only cookies, validating them, and populating the Spring Security context for authenticated administrator sessions. The

`AdminUserDetailsService` integrates with Spring Security by retrieving administrator account information from the `AdminUserRepository` during the authentication process.

Overall, the package structure of the Al-Muneer Online Portal follows established Spring Boot architectural practices by separating responsibilities into dedicated layers. Controllers manage request handling, services encapsulate business logic, repositories provide data access, and models represent persistent entities. Meanwhile, the configuration, utility, and security packages address cross-cutting concerns that support the operation of the entire application. This organization improves code maintainability, enhances scalability, and facilitates future system extensions while ensuring that dependencies flow in a structured manner from controllers to services, services to repositories, and repositories to domain models.

4.8 Interaction Viewpoint

The Interaction Viewpoint defines strategies for interaction among design entities (components, classes, subsystems), regarding why, where, how, and at what level actions occur. It illustrates the dynamic behavior of the system, showing how different parts of the system collaborate to achieve specific functionalities.

4.8.1 Design Concerns

The primary design concerns for the Interaction Viewpoint are:

- **Dynamic Behavior:** How system components and objects interact over time to accomplish tasks.
- **Collaboration:** Understanding the sequence of messages or calls exchanged between different parts of the system.
- **Responsibility Allocation:** Clarifying which component is responsible for initiating an interaction and which component responds.

- **Use Case Realization:** Showing how the interactions realize the use cases defined in the SRS.

The functionalities of the system are given by the help of sequence diagrams. Moreover, it defines strategies for interaction among entities. For the Al-Muneer Online Portal, sequence diagrams are used to model these interactions.

4.8.2 Design View

Detailed sequence diagrams illustrating the step-by-step interactions for each specific use case (UC001 through UC016), involving actors, frontend components, backend controllers, services, and database interactions, have already been provided in the **Software Requirements Specification (SRS) document, Section 2.3: Use Case Details**. These SRS sequence diagrams provide a granular view of how each functional requirement is realized through object interactions.

This section of the SDD will therefore focus on providing a higher-level view of interactions between the major subsystems identified in the Composition Viewpoint (Section 5.3, Figure 5.2). These diagrams illustrate how these coarse-grained components collaborate for key overarching scenarios.

a) SD001: High-Level Sequence Diagram for Visitor Submitting a Booking Inquiry and Initial Admin Notification

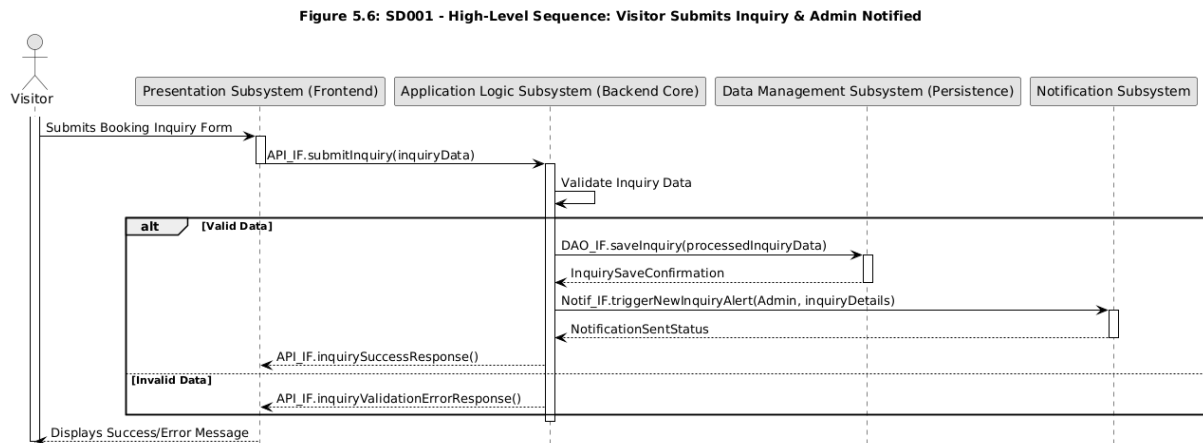


Figure 5.6: SD001 - High-Level Sequence: Visitor Submits Inquiry & Admin Notified

Description of SD001:

The Visitor initiates the process by submitting the booking inquiry form via the Presentation Subsystem (Frontend). The Frontend forwards this request to the Application Logic Subsystem via its API. The Application Logic Subsystem first validates the received data. If valid, it instructs the Data Management Subsystem to save the inquiry. Upon successful saving, the Application Logic Subsystem then triggers the Notification Subsystem to alert the Administrator about the new inquiry. Finally, a response (success or validation error) is sent back to the Frontend, which then updates the Visitor.

b) SD002: High-Level Sequence Diagram for Administrator Managing Payment Proof Status

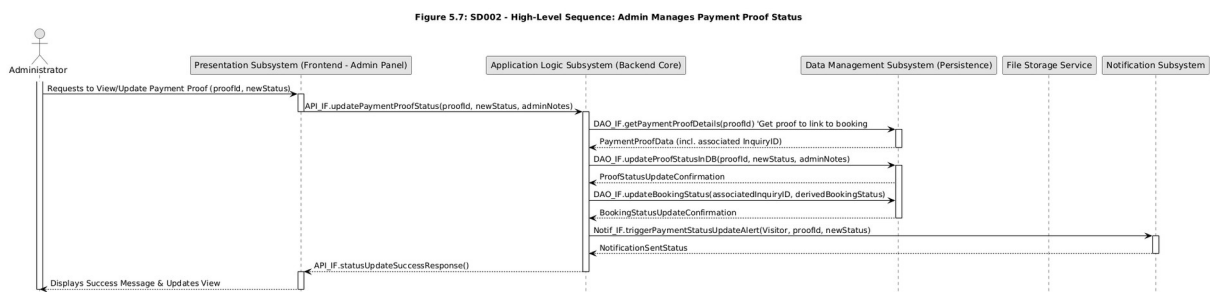


Figure 5.7: SD002 - High-Level Sequence: Admin Manages Payment Proof Status

Description

of

SD002:

The Administrator, via the Admin Panel (Presentation Subsystem), initiates an update to a payment proof's status. The request is sent to the Application Logic Subsystem. The AppLogic first retrieves details of the payment proof (and potentially the associated booking inquiry) from the Data Management Subsystem. It then instructs the Data Management Subsystem to update the status of the payment proof and possibly the linked booking inquiry. After successful database updates, the AppLogic triggers the Notification Subsystem to inform the Visitor of their payment status update. A success response is then relayed to the Admin Panel, which updates the Administrator's view. The File Storage Service would have been involved earlier when the admin viewed the actual proof image, orchestrated by the AppLogic.

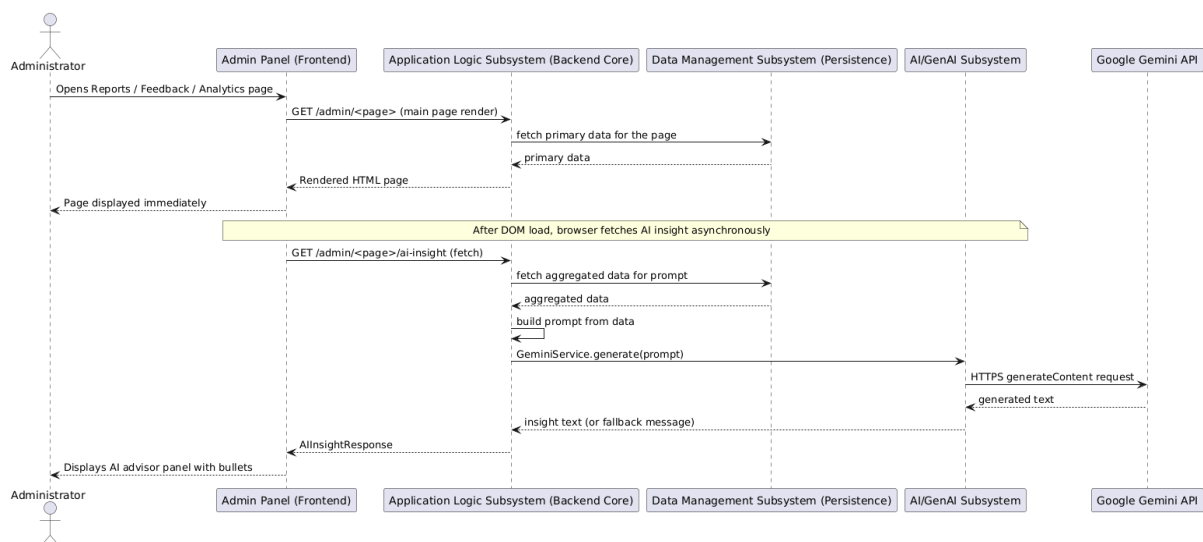


Figure 5.8: SD003 - High-Level Sequence: Asynchronous AI Advisor Panel

Description of SD003:

When the Administrator opens a page that includes an AI advisor (Reports, Feedback, or Analytics), the Backend Core first renders the page using only local data. The AI insight is never fetched during the initial render. After the browser finishes loading the DOM, a separate asynchronous `fetch()` requests the insight endpoint (`/admin/reports/ai-insight`, `/admin/feedback/ai-insight`, or `/admin/analytics/ai-insight`). The corresponding controller aggregates the relevant data, constructs a plain-text prompt, and delegates to `GeminiService`. `GeminiService` calls the Google GenAI Java SDK, which communicates

with the Gemini API over HTTPS. If the SDK call succeeds, the generated text is returned to the browser and formatted as styled bullet points. If any failure occurs (network issue, invalid API key, empty response), `GeminiService` catches the exception and returns a human-readable fallback message such as "AI summary unavailable — API call failed." This design guarantees that AI failures never block page rendering or degrade the core user experience.

State Machine Diagram for BookingInquiry Entity

4.9 State Dynamic Viewpoint

Reactive systems and systems whose internal behavior is of interest use this viewpoint. System dynamics including modes, states, transitions, and reactions to events are described here. This section focuses on the behavior and states of key entities within the Al-Muneer Online Portal, illustrated via state transition diagrams. This informs designers, developers, and testers about the dynamic view of critical parts of the system.

4.9.1 Design Concerns

The primary design concerns for the State Dynamic Viewpoint are:

- **Entity Lifecycles:** Understanding the different states an entity can be in throughout its lifecycle.
- **State Transitions:** Identifying the events or conditions that cause an entity to transition from one state to another.
- **Actions/Activities:** Specifying any actions or activities that occur when entering a state, exiting a state, or during a transition.
- **System Behavior:** Clarifying how key entities respond to various system events and user interactions.

In this section, system behavior and states of the system are given via the state transition diagram. Designers, developers and testers are informed about dynamic view of the system

which includes its states, transitions, and the events that trigger these transitions for key entities.

4.9.2 Design View

State transition diagrams are included here for key entities in the Al-Muneer Online Portal whose behavior involves distinct and significant state changes.

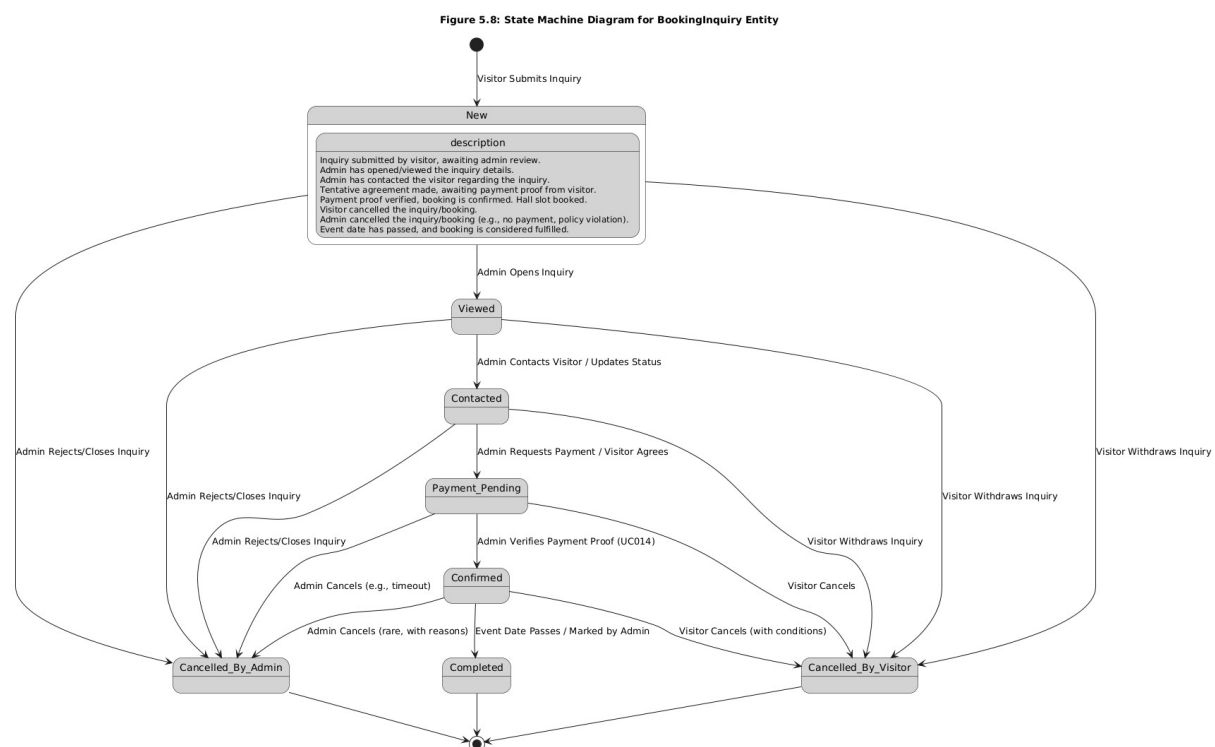


Figure 5.9: State Machine Diagram for BookingInquiry Entity

Description of BookingInquiry States:

The BookingInquiry entity transitions through several states from its creation to its resolution. It begins in a 'NEW' state upon submission. An administrator action moves it to 'VIEWED' and then potentially 'CONTACTED'. If discussions lead to a tentative agreement, it moves to 'PAYMENT_PENDING'. Successful verification of a

`PaymentProof` (UC013) transitions it to `CONFIRMED` and also marks the linked `AvailabilitySlot` as `BOOKED`. The inquiry can be `CANCELLED\_BY\_ADMIN` or `CANCELLED\_BY\_VISITOR` at various stages; visitor self-cancellation is only allowed when no payment proof is attached, and it frees the slot back to `AVAILABLE`. After the event date, an administrator may mark a `CONFIRMED` booking as `COMPLETED`.

Figure 5.9: State Machine Diagram for PaymentProof Entity

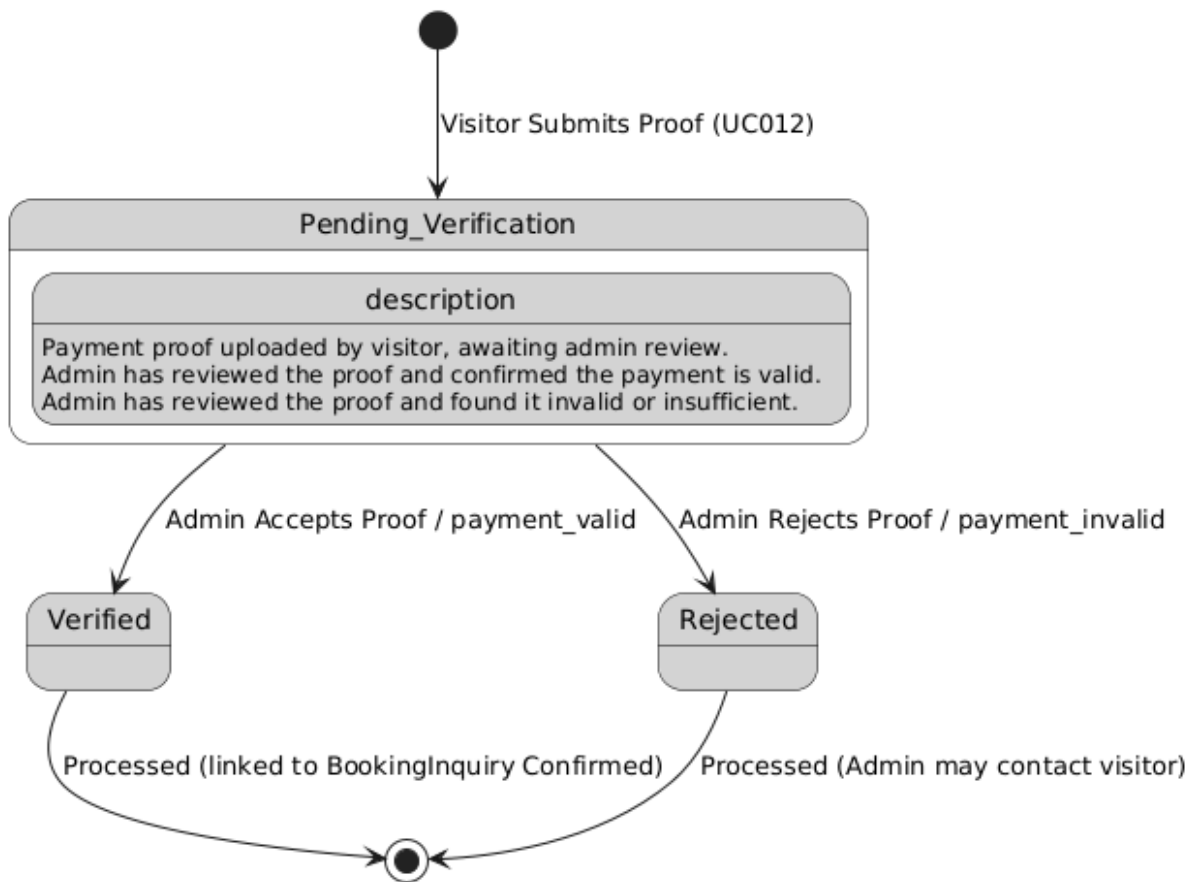


Figure 5.10: State Machine Diagram for PaymentProof Entity

Description of PaymentProof States:

The PaymentProof entity has a simpler lifecycle. Upon submission by a visitor, it enters the Pending_Verification state. The administrator then reviews it. If the proof is deemed valid, its state changes to Verified. If the proof is insufficient or invalid, its state changes to Rejected. Both Verified and Rejected are terminal states for the proof itself, triggering further actions related to the associated BookingInquiry.

These state machine diagrams illustrate the lifecycle and dynamic behavior of key entities within the Al-Muneer Online Portal, providing clarity on how they respond to system events and user interactions.

4.10 Algorithm Viewpoint

The Algorithm Viewpoint provides detailed design descriptions of operations, such as methods and functions, including the internal details and logic of each design entity. It provides details needed by programmers and analysts of algorithms in regard to time-space performance and processing logic prior to implementation, and to aid in producing unit test plans.

4.10.1 Design Concerns

The primary design concerns for the Algorithm Viewpoint are:

- **Processing Logic:** Detailing the step-by-step logic for specific operations or functions within the system.
- **Data Manipulation:** How data is transformed, calculated, or processed by algorithms.
- **Decision Points:** Clarifying conditions and branching logic within operations.
- **Clarity for Implementation:** Providing sufficient detail for developers to implement the algorithms correctly.

For the Al-Muneer Online Portal, the core processing logic for most functionalities aligns closely with the realization of the use cases.

4.10.2 Design View

The detailed step-by-step processing logic for the majority of user-facing and administrative operations in the Al-Muneer Online Portal is extensively documented through the Activity Diagrams provided for each Use Case (UC001 through UC016) in the Software Requirements Specification (SRS) document, Section 2.3: Use Case Details. These activity diagrams illustrate the flow of actions, decision points, and interactions required to fulfill each specific use case. They serve as the primary algorithmic design for those operations.

For example:

- The algorithm for a **Visitor Submitting a Booking Inquiry (UC005)** is detailed in its corresponding activity diagram in SRS Section 2.3.5. This includes steps for form access, data input, validation, saving the inquiry, and triggering notifications.
- The algorithm for an **Administrator Managing Payment Status (UC013)** is detailed in its activity diagram in SRS Section 2.3.14. This covers viewing proofs, offline verification, updating status in the system, and notifying the visitor.

Within the backend (Spring Boot application), this logic is implemented within the Service layer classes (e.g., 'BookingInquiryServiceImpl', 'PaymentProofServiceImpl'). These service methods encapsulate the algorithms depicted in the SRS activity diagrams.

Key algorithmic considerations within these services include:

- **Data Validation:** All user inputs (from inquiry forms, admin content management, etc.) will undergo server-side validation logic as per defined business rules (e.g., mandatory fields, data formats, value ranges) before processing.
- **Availability Checking Logic:** When a visitor checks availability or an admin updates the calendar, the AvailabilityService will implement logic to query AvailabilitySlot entities for specific dates or date ranges and determine their status.
- **Pricing Calculation (if dynamic options are implemented):** If pricing involves selectable add-ons, the PricingService would contain logic to calculate total estimates based on selected base tiers and additional options.
- **Report Generation Logic:** The ReportService will contain logic to fetch data from various entities (e.g., BookingInquiry, Feedback) based on admin-specified

parameters (like date ranges) and then aggregate/format this data into a simple, readable report structure for display. This will primarily involve structured database queries and data mapping.

- **File Handling for Payment Proofs:** The `PaymentProofService` (utilizing a `FileUploadUtil`) will implement the logic for receiving an uploaded file, validating its type and size, securely storing it in the designated file storage, and associating its path and metadata with the corresponding `BookingInquiry`.

Additional algorithmic considerations specific to the implemented system include:

- **Reference Code Generation:** When a `BookingInquiry` is persisted, `@PrePersist` generates a random 9-digit `Long` value in the range `[100_000_000, 999_999_999]` using `ThreadLocalRandom`. This value is stored in a 150-day HTTP-only cookie named `inq_ref` so the visitor can retrieve or cancel their inquiry without authentication.
- **Status Cascade on Payment Proof Verification:** When an administrator marks a `PaymentProof` as `VERIFIED`, the service layer atomically updates the linked `BookingInquiry.status` to `CONFIRMED` and the linked `AvailabilitySlot.status` to `BOOKED`. Rejection leaves the inquiry in its current state for manual follow-up.
- **Visitor Self-Cancellation:** The cancellation service first checks whether the inquiry has any attached payment proofs. If proofs exist, cancellation is rejected with an `IllegalStateException`. Otherwise, the inquiry status becomes `CANCELLED_BY_VISITOR` and the reserved slot returns to `AVAILABLE`.
- **AI Prompt Construction and Graceful Fallback:** Each AI controller builds a plain-text prompt that includes aggregated figures (e.g., conversion rate, cancellation rate, funnel counts, low/high rating samples). The prompt explicitly requests a fixed number of bullet points tied to real numbers. `GeminiService.generate()` wraps the SDK call in a try/catch block; if the response is empty, malformed, or the call throws,

a fallback string such as "AI summary unavailable — API call failed." is returned. This guarantees that AI failures are non-blocking.

- WhatsApp Deep Link Generation: Client-side JavaScript reads the selected `NotificationTemplate.templateText`, replaces placeholders with values from the current inquiry/payment record, normalises the visitor phone number with the configured country code, URL-encodes the message, and opens `'https://wa.me/<number>?text=<message>'` in a new tab. No server-side WhatsApp API is used.

in SRS Section 2.3.14. This covers viewing proofs, offline verification, updating status in the system, and notifying the visitor. The detailed design of these specific algorithms is implemented within the respective Java service methods, leveraging Spring Boot and JPA functionalities for data access and transaction management.

5. User Interface Design

This section describes the user interface design for the Al-Muneer Online Portal. It provides an overview of the UI from the user's perspective and conceptual layouts for key screens. The design aims for simplicity, usability, and a professional appearance appropriate for Al-Muneer Hall, with a primary focus on the Arabic language and cultural context of Yemen.

5.1 Overview of User Interface

The Al-Muneer Online Portal will feature a responsive web-based Graphical User Interface (GUI) accessible via standard desktop and mobile web browsers. The overall design philosophy emphasizes clarity, ease of navigation, and efficient task completion for both Visitors and the Administrator.

5.1.1 Visitor Interface:

Navigation will be managed through a clear, persistent main navigation menu, likely a header bar, offering access to key sections such as Home, About Us/Venue Info, Gallery, Availability, Pricing, Booking Inquiry, FAQ, Feedback, and Contact. Pages will generally adopt a clean layout, comprising a header, a main content area, and a footer that includes contact information and copyright details. Visuals, specifically images of the hall, will be incorporated judiciously to enhance the portal's appeal. The primary language for content and navigation will be Arabic, with potential for English equivalents for key navigation labels if deemed necessary. Interactivity will be facilitated through forms for inquiries, payment proof submission, and feedback. The availability calendar will be interactive, and image galleries will allow users to view larger versions of images. Users will receive clear confirmation messages for actions like inquiry submission or feedback, along with informative error messages.

5.1.2 Administrator Interface (Admin Panel):

Access to the admin panel will be secured via a login page. Navigation within the admin panel will be provided by a sidebar granting access to management modules: Dashboard (overview), Venue Info, Media Gallery, Gallery Labels, Availability Calendar, Pricing, Booking Inquiries, Payment Proofs, Customer Feedback, Reports, Analytics, and Notification Templates. The layout will typically feature a dashboard presenting summary cards (new inquiries, pending proofs, unreviewed feedback, visits today/last 7 days, average rating), followed by dedicated pages for managing specific data types, often employing tables, filter bars, and forms. The Reports and Analytics pages include Chart.js visualisations and an asynchronous AI advisor card that loads after the page renders. The primary language of the administrator interface will be Arabic to ensure ease of use for Mr. Almunajid. Interactivity in the admin panel will include data entry forms, tables with client-side search and status filters, action buttons (Save, Update, Delete, View, Verify, Reject, Mark Reviewed), file upload widgets, and a WhatsApp template selector that generates a `wa.me` deep link on inquiry/payment detail pages.

AI Advisor Panels:

Three admin pages include optional AI advisor cards:

- **Reports Page:** Displays an "AI Business Report Advisor" card showing 3 bullet points derived from the current period's inquiry, payment, and feedback figures.
- **Feedback Page:** Displays an "AI Feedback Advisor" card highlighting what guests love, the most urgent complaint, and one concrete action.
- **Analytics Page:** Displays an "AI Traffic Funnel Advisor" card analysing Home → Gallery → Pricing → Inquiry visit counts and suggesting one conversion improvement.

Each card initially shows a spinner and loads its content asynchronously via `fetch()`. If the Gemini API is unavailable or unconfigured, the card displays a graceful fallback message instead of crashing the page.

5.2 Screen Images (English Version)

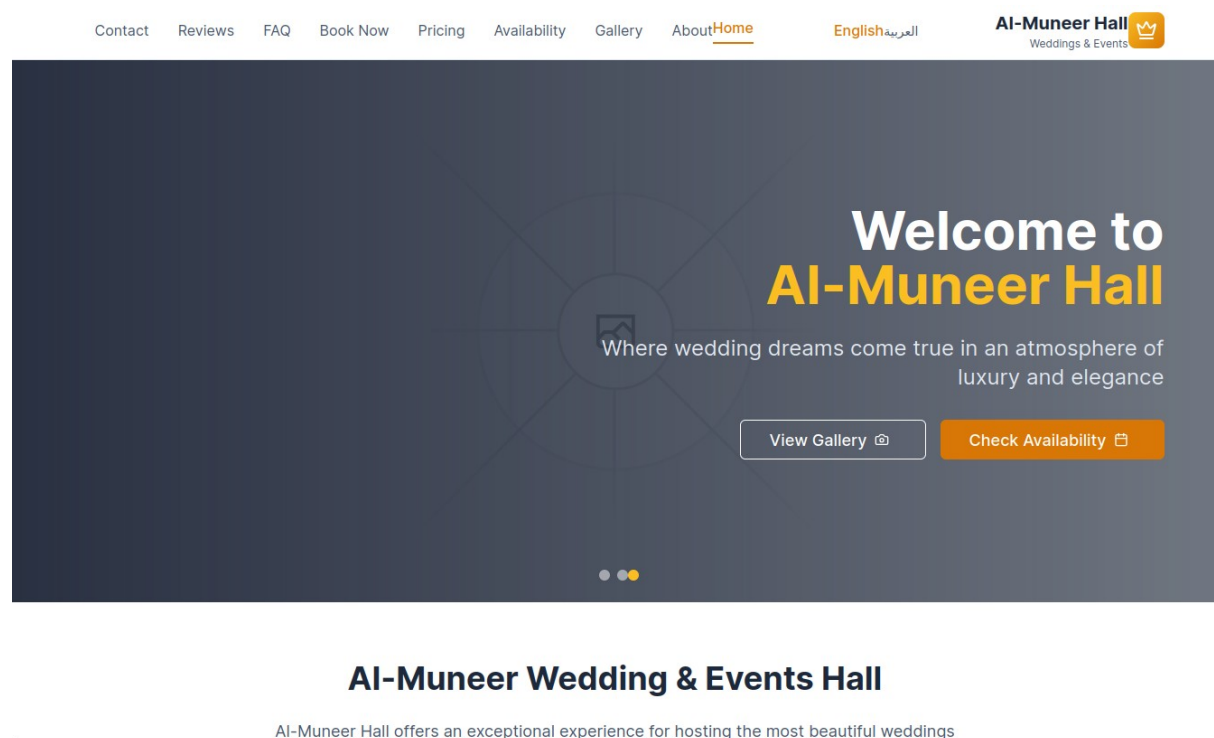


Figure 6.1: Visitor Interface - Homepage

This figure displays the homepage of the visitor interface, showcasing the initial impression and primary navigation elements available to users exploring the AI-Muneer Online Portal.

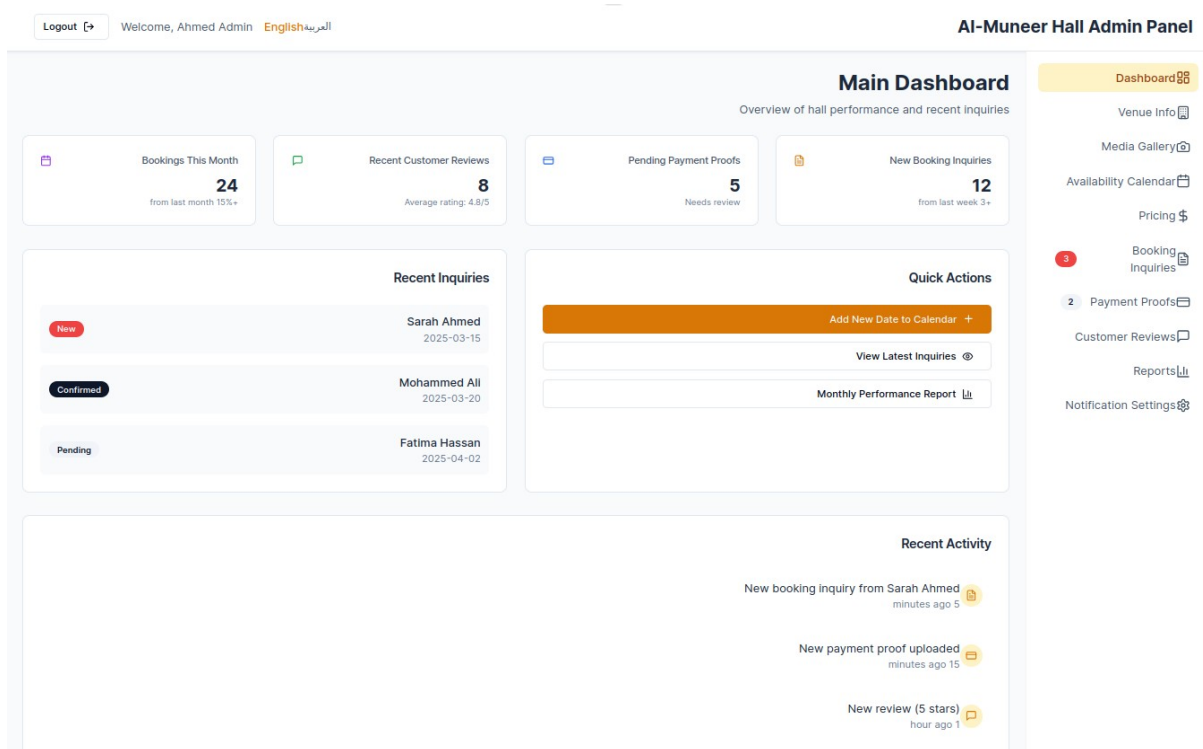


Figure 6.2: Administrator Interface - Admin Dashboard

This figure illustrates the administrator dashboard, providing an overview of key metrics and quick access to different management functions within the AI-Muneer Online Portal.

Logout>Welcome, Ahmed AdminEnglishالعربية

AI-Muneer Hall Admin Panel

Manage Booking Inquiries

View and manage all booking inquiries from customers

To Date

mm / dd / yyyy

From Date

mm / dd / yyyy

Status

All Statuses

Filter Results

Search

...Search by name or ID

Apply Filter

Booking Inquiries List (5 inquiries)

Actions	Status	Submission Date	Event Date	Contact Number	Visitor Name	Inquiry ID
	New	2025-01-15	2025-03-15	456 123 777 967+966	Sarah Ahmed Mohammed sara@email.com	BK-2025-001
	Confirmed	2025-01-14	2025-03-20	567 234 777 967+966	Mohammed Ali Hassan mohammed@email.com	BK-2025-002
	Pending	2025-01-13	2025-04-02	678 345 777 967+966	Fatima Hassan Ahmed fatima@email.com	BK-2025-003
	Confirmed	2025-01-12	2025-04-10	789 456 777 967+966	Ahmed Mohammed Ali ahmed@email.com	BK-2025-004
	Cancelled	2025-01-11	2025-04-18	890 567 777 967+966	Noor Al-Din Yahya noor@email.com	BK-2025-005

Next >

3

2

1

Previous <

Showing 1-5 of 12 inquiries

Dashboard

Venue Info

Media Gallery

Availability Calendar

Pricing

Booking Inquiries

Payment Proofs

Customer Reviews

Reports

Notification Settings

Figure 6.3: Administrator Interface - Manage Inquiries Page

This figure depicts the "Manage Inquiries" page within the administrator interface, showing how booking inquiries are listed and can be managed by the administrator.

6. Appendices

Appendix A: (Placeholder for Future Detailed API Specifications)

management procedures, or specific interface contracts that are too extensive for the main design body, such details would be provided here. Currently, the design allows for such integrations, but the selection and detailed design of the interface with a specific provider is part of the implementation phase.

Appendix B: (Placeholder for Third-Party Service Integration Details)

If specific third-party services are selected and their integration requires detailed configuration steps, API key management procedures, or specific interface contracts that are too extensive for the main design body, such details would be provided here. Currently, the design allows for such integrations, but the selection and detailed design of the interface with a specific provider is part of the implementation phase.